PR Rechnernetze und Netzwerkprogrammierung 621.801/621.802

Institut für Informationstechnologie (ITEC) · WS 2026/2027

Übungsblatt 03

POP3 und HTTP mit Python Sockets

Ü 3.1 Post Office Protocol 3 (POP3)

Machen Sie sich mit POP3 vertraut. Verwenden Sie RFC 1939 und untersuchen Sie, wie ein Client sich bei einem Email-Server anmeldet und Nachrichten abruft.
Beantworten Sie:

- Wie ist POP3 Arbeitsablauf aufgebaut?
- Ist POP3 sicher?
- Was ist der Unterschied zwischen single-line und multi-line response? Vergleichen Sie mit IMAP.

Ü 3.2 Python Sockets: POP3-Client

Implementieren Sie einen einfachen POP3-Client in Python. Ihr Client soll sich anmelden, eine Liste aller Nachrichten anfordern und eine Nachricht abrufen. Achten Sie auf ordnungsgemäßen Verbindungsabbau. Verwenden Sie die Python-Standardbibliothek und Socket-Kommunikation. Testen Sie mit einem geeigneten POP3-Server (ohne Verschlüsselung). Alternativ können Sie https://www.stunnel.org/ verwenden, um eine sichere Verbindung mit einem POP3-Server Ihrer Wahl herzustellen. Alternativ können Sie auch einen lokalen POP3-Server hosten; wir stellen hierfür eine Implementierung zur Verfügung: https://github.com/kurthorvath/GoPop3Server.

Ü 3.3 Python Sockets: POP3-Server

Implementieren Sie einen POP3-Server mit minimaler Funktionalität, mit dem ein Email-Client Ihrer Wahl zusammenarbeitet. Nutzen Sie die bereitgestellte Python-Datenbankklasse **SampleDataBase.py** aus dem Moodle-Kurs/Github. Ziel ist, dass die Nachrichten daraus in der Inbox eines Email-Clients angezeigt werden (*popMan*, *Thunderbird*). Mehrere Benutzer sollen gleichzeitig verbunden sein können; verwenden Sie dafür pro Benutzer einen Thread. Testen Sie Client und Server mit tatsächlichen Implementierungen, beschreiben Sie den Ablauf inkl. Screenshots. (siehe: RFC 1939 und RFC 2449)

Ü 3.4 Python Sockets: HTTP-Server (single-thread)

Implementieren Sie einen HTTP-Server, der Anfragen von Clients in einem einzigen Thread behandelt. Die Funktionalität soll sich auf HTTP/0.9 und das Ausliefern einzelner Dateien beschränken. **TCP-Port (80)** und Basisverzeichnis (document root) sollen beim Start über Kommandozeilenparameter angegeben werden. Bei Fehlern soll die Verbindung gemäß der ursprünglichen Aufgabenstellung ohne Rückmeldung geschlossen werden. Für ein angefordertes Verzeichnis soll index.html zurückgegeben werden. Verwenden Sie die bereitgestellten Testdaten ***documentRoot.zip.***
Testen Sie mit unterschiedlichen browsern. Werden die Daten (Webseite) richtig angezeigt?

Ü 3.5 Python Sockets: HTTP-Server (multi-thread)

Erweitern Sie den Server aus Ü 3.4: Jede Anfrage soll durch einen neuen Thread behandelt werden. Unterstützen Sie HTTP/1.0 und persistente Verbindungen. Ergänzen Sie eine Logging-Komponente, die Zeitpunkt, Request-Typ, IP-Adresse und TCP-Port des Clients protokolliert. Das Logging soll zentral durch einen eigenen Thread erfolgen, der das Zugriffsprotokoll in Abständen von 5 Sekunden ausgibt. Sorgen Sie für geeignete Synchronisation.